# FUNCTIONAL COVERAGE OF THE TESTS

*Computer Engineering, A. Y. 2025/2026*
*Samuele Randazzo, Gabriele Pirani, Riccardo Pizzaleo, Claudio Putrone*
*Final Exam (Software Engineering)*

POLITECNICO
MILANO 1863

## 1. Introduction

This document describes the functional coverage of the tests developed for the Software Engineering Final Exam. The goal is not to report only a percentage of lines of code executed, but to clarify which system features have been verified, which design responsibilities are protected from regressions and how the test suite supports the correctness of the model, the game rules and the main application flows.

The test suite consists of a total of **124 tests**, spread over **23 test classes**. The results of the tests in the project indicate **0 failures** and **0 errors**. The tests are mainly focused on game logic, model rules, cards, events, game states, data loading, persistence and some integration scenarios.

## 2. Method of analysis

The analysis was carried out considering:

- the tests in the src/test/java directory;
- the Maven configuration of the project in pom.xml;
- the structure of the application code in src/main/java;
- the game resources present in src/main/resources.

The document therefore adopts a **functional coverage** perspective: the test classes are analyzed in relation to the game scenarios and the application responsibilities actually verified.

## 3. Functional distribution of tests

From a functional point of view, the tests are divided into the following macro-categories.

| Functional category | Number of tests |
|---|---|
| Main game logic, states and simulations | 41 |
| Board, offer tracks and turn order | 24 |
| Cards, event-cards and tribes | 21 |
| Event effects | 21 |
| Data loading from JSON resources | 8 |
| Controller, persistence and recovery | 9 |
| **Total** | **124** |

This breakdown shows that coverage is mainly geared towards the correctness of the application domain, i.e., the part of the code that implements the rules of the game. The suite therefore focuses on the components that have the greatest impact on the state of the game, scores, player resources and the progression of the game flow.

## 4. Coverage by functional macro-category

### 4.1 Main Game Logic, States, and Simulations

This is the largest category in the suite and includes **41 tests**. It includes the GameTest, GameFlowTest, GameAdvancedTest, FullGameSimulationTest, GameStateTest, and TotemPlacementStateTest classes.

The tests cover the main features of the match:

- creation of the game with minimum and maximum number of players;
- rejection of invalid configurations, such as zero player list or number of players out of bounds;
- management of the current turn and advancement to the next player;
- return to the first player after the last player in the round;
- increase of the round and reset of the current player;
- initialization of the game and transition to the totem placement state;
- default behavior of non-enabled states;
- placement of totems during the initial state;
- change of status when all players have placed their totem;
- transition of era and loading of buildings of the next era;
- deterministic game simulations to verify consistency of the global state.

This category is particularly relevant because it tests the main cycle of the game. Testing allows you to check that the game is initialized correctly, that the progression of rounds and rounds is consistent, and that state transitions respect the expected behavior.

### 4.2 Board, offer track and turn order

This category includes **24 tests**, which are distributed between BoardTest, OfferTileTest, OfferTrackTest, TurnOrderSlotTest, and TurnOrderTileTest.

The tests verify:

- filling of board rows;
- removal of cards from the top row;
- management of invalid indexes during card selection;
- end-of-round cleanup, moving cards from the top row to the bottom row;
- Updating the era and loading new buildings;
- replacement of the building deck;
- management of empty decks;
- availability of offer tiles;
- placement and removal of totems;
- protection against multiple occupation of a slot;
- Calculation of the order of the players for the next round.

The board and tracks are central elements of the regulations. Their coverage contributes significantly to the functional correctness of the project, since many game actions depend on the correct management of rows, tiles and turn order.

### 4.3 Cards, event-cards and tribes

This category contains **21 tests**, related to CardTest, EventCardTest, and TribeTest.

Verified features include:

- validation of common card parameters, how it was and minimum number of players;
- management of invalid parameters in constructors;
- recognition of event cards;
- invocation of the effect associated with an event card;
- handling exceptions when resolving an event;
- updating the counts of characters in the tribe;
- hunters' bonuses on food;
- shamanic attributes;
- calculations related to builders;
- management of invention icons;
- bonuses from buildings linked to the tribe.

This part of the suite protects the rules related to the growth of the tribe and the application of card effects. The coverage is good compared to the logic implemented, because it includes both item construction and effects on the player's state.

### 4.4 Event Effects

Event effects are covered by **21 tests**: HuntTest, SustenanceTest, CavePaintingsTest, and ShamanicRitualTest.

The tests verify:

- assignment of prestige points based on hunters;
- absence of points when no hunters are present;
- application of bonuses of buildings related to hunting;
- penalty for insufficient food during sustenance;
- absence of penalties when the food is sufficient;
- discounts from collectors and livelihood buildings;
- penalties and bonuses of cave paintings;
- management of winners, losers and draws in shamanic ritual;
- interaction between shamanic ritual and special attributes;
- handling invalid or null inputs.

This category is important because events directly change players' score and resources. The tests cover the main functional branches of the implemented effects and reduce the risk of regressions in the most sensitive rules of the game.

### 4.5 Loading Data from JSON Resources

Data loading is covered by **8 tests** in the GameDataLoaderTest class.

The tests verify:

- loading tribe cards from JSON files;
- inclusion of event cards in the deck;
- correct number of event cards loaded;
- loading of buildings of eras 1, 2 and 3;
- reduced offer track loading for two players;
- Loading of the complete Offer Track for more than two players.

These tests are relevant because they verify that the code is consistent with the external data actually used by the game. An incorrect change to the JSON files can then be detected through the test suite.

### 4.6 Controller, Persistence, and Recovery

This category contains **9 tests**, which are distributed between GameControllerTest, PersistenceControllerIntegrationTest, PersistenceManagerTest, and LobbyRecoveryTest.

The verified features are:

- management of the selection of a tile by means of a controller;
- tile occupation and shift advancement;
- saving after totem placement and card choice;
- creation of the save file;
- loading a saved game;
- Reset of rounds, era, players, resources, and current player;
- restoration of boards and offer tracks sufficiently to continue the game;
- deletion of the save;
- Lobby recovery when saved players reconnect.

These tests test for some important integration scenarios. In particular, they check that the state of the game can be saved, reloaded and resumed, in accordance with the advanced persistence functionality.

## 5. Detail of test classes

The following table shows the details of the test classes considered in the analysis, indicating for each the number of tests and the functional area covered.

| Test class | Tests | Coverage area |
|---|---|---|
| GameTest | 18 | Game construction, validations, leaderboard and events |
| GameFlowTest | 8 | Turns, rounds and game progression |
| GameAdvancedTest | 4 | Era transitions and advanced events |
| FullGameSimulationTest | 3 | Deterministic simulations of full games |
| GameStateTest | 4 | General game state behaviour |
| TotemPlacementStateTest | 4 | Totem placement state |
| BoardTest | 9 | Board management, card rows and era changes |
| OfferTileTest | 3 | Offer tiles |
| OfferTrackTest | 3 | Offer track |
| TurnOrderSlotTest | 4 | Turn order slots |
| TurnOrderTileTest | 4 | Turn order tiles |
| CardTest | 3 | Base card class |
| EventCardTest | 7 | Event card management and resolution |
| TribeTest | 11 | Tribes, characters and related bonuses |
| HuntTest | 5 | "Hunt" event effect |
| SustenanceTest | 6 | "Sustenance" event effect |
| CavePaintingsTest | 5 | "Cave Paintings" event effect |
| ShamanicRitualTest | 5 | "Shamanic Ritual" event effect |
| GameDataLoaderTest | 8 | Data loading from JSON files |
| GameControllerTest | 1 | Tile selection via controller |
| PersistenceControllerIntegrationTest | 1 | Controller and persistence mechanism integration |
| PersistenceManagerTest | 4 | Game state saving and loading |
| LobbyRecoveryTest | 3 | Lobby recovery and player reconnection |
| **Total** | **124** | **Automated tests passing successfully** |

## 6. Functional structure of the suite

The structure of the suite can be summarized by grouping the test classes according to the package or verified responsibility:

```
src/test/java/it/polimi/ingsw/
|-- model/
| |-- GameTest
| |-- GameFlowTest
| |-- GameAdvancedTest
| |-- FullGameSimulationTest
| |-- BoardTest
| |-- CardTest
| |-- EventCardTest
| |-- TribeTest
| |-- EventEffects/
| | |-- HuntTest
| | |-- SustenanceTest
| | |-- CavePaintingsTest
| | |-- ShamanicRitualTest
| |-- states/
| |-- GameStateTest
| |-- TotemPlacementStateTest
|-- controller/
| |-- GameControllerTest
|-- factories/
| |-- GameDataLoaderTest
|-- persistence/
| |-- PersistenceManagerTest
| |-- PersistenceControllerIntegrationTest
|-- server/
    |-- LobbyRecoveryTest
```

This representation highlights how the largest number of tests is concentrated on the domain package, consistent with the objective of verifying the main rules of the game and the consistency of the internal state of the game.

## 7. Functional Coverage Assessment

Coverage is high for the domain package and for classes that implement the main rules of the game. In particular, the following responsibilities are well covered:

- batch creation and validation;
- progression of turns, rounds and eras;
- management of the board and tiles;
- cards, tribes, and event effects;
- loading of game assets;
- basic save, load and recovery.

Overall, the suite is adequate to verify the correctness of the rules of the game and the central components of the model. The choice to focus the tests on domain classes is consistent with the role of the model in the MVC architecture of the project: it is in fact in this part of the code that scores, resources, events, cards, boards and game progress are managed.

## 8. Conclusion

The test suite significantly covers the core capabilities of the project. With **124 tests** spread over **23 classes**, the functional coverage is particularly solid on the model and game rules: game creation, turns, boards, cards, tribes, events, data loading, persistence and basic recovery.

The document therefore shows that the verification is not limited to isolated controls on manufacturers, but includes broader functional scenarios and integration cases related to the overall behavior of the system.